POLITECHNIKA WROCŁAWSKA

WYDZIAŁ INFORMATYKI I TELEKOMUNIKACJI

PROJEKTOWANIE EFEKTYWNE ALGORTYMÓW

TSP – Problem Komiwojażera

ZADANIE 2 Algorytm Branch and Bound

Grupa:

Wtorek 9:15, Grupa nr. 3

Autor:

Karolina Obiedzińska

Indeks:

273034

Prowadzący

dr inż. Marcin Łopuszyński

Spis treści

1. **Wstęp** 3
   - 1.1 **Opis rozpatrywanego problemu** 3
   - 1.2 **Oszacowanie złożoności problemu** 3
2. **Przykład praktyczny** 4
   - 2.1 Algorytm Branch and Bound (przy przeszukiwaniu wszerz BFS) 4
   - 2.2 Algorytm BFS 4
   - 2.3 Algorytm DFS 5
   - 2.4 Algorytm Branch and Bound - Best-First Search 5
6. **Plan eksperymentu** 5
   - a. **Rozmiary używanych struktur danych** 5
   - b. **Sposób generowania danych** 6
   - c. **Metoda pomiaru czasu** 6
   - d. **Warunki badawcze:** 6
7. **Wyniki eksperymentu** 7
   - a. **Macierze symetryczne** 7
   - b. **Macierze asymetryczne** 9
8. **Wnioski** 11
   - a. **Tabela symetryczna** 11
   - b. **Tabela asymetryczna** 11
   - c. **Porównanie Depth First Search i Best-First Search dla obu typów macierzy** 11
   - d. **Efektywność poszczególnych algorytmów** 11
   - e. **Możliwe poprawki** 11
9. **Kod źródłowy** 12
10. **Bibliografia:** 12

1. Wstęp

1.1 Opis rozpatrywanego problemu

Problem komiwojażera[1](ang. Traveling Salesman Problem, TSP) to jedno z klasycznych zagadnień optymalizacyjnych w teorii grafów i informatyce. Polega na znalezieniu najkrótszej możliwej trasy, która pozwala komiwojażerowi odwiedzić każde z zadanych miast dokładnie raz i wrócić do miasta startowego. Jest to problem NP-trudny, co oznacza, że nie istnieje znany algorytm, który rozwiązuje go w czasie wielomianowym dla dużych zbiorów danych. Z tego powodu opracowano różne podejścia i heurystyki do jego rozwiązywania.

Algorytm Branch and Bound (podziału i ograniczeń) jest metodą optymalizacyjną, stosowaną do rozwiązywania problemów decyzyjnych, takich jak problemy kombinatoryczne czy optymalizacji globalnej. Algorytm eksploruje przestrzeń rozwiązań, dzieląc ją na mniejsze podproblemy (gałęzie) i ograniczając rozważanie tych, które nie mogą prowadzić do optymalnego rozwiązania.[2]

- ***Algorytm DFS (przeszukiwanie w głąb)*** to metoda eksploracji grafu, która podąża w głąb od wierzchołka startowego aż do napotkania ślepej uliczki, po czym cofa się do poprzedniego punktu rozgałęzienia.[2]
- ***Algorytm BFS (przeszukiwanie wszerz)*** to metoda przeszukiwania grafów, która eksploruje graf w kolejności poziomów (warstw) od wierzchołka startowego. [2]
- ***Algorytm Best-First Search (przeszukiwanie "najpierw najlepszych")*** to heurystyczna metoda eksploracji grafu, która wybiera wierzchołki do rozwinięcia na podstawie oceny ich potencjału, zwykle przy użyciu funkcji heurystycznej. [2]

1.2 Oszacowanie złożoności problemu

W najgorszym przypadku algorytm BFS musi odwiedzić wszystkie wierzchołki oraz wszystkie krawędzie w grafie. W związku z tym jego złożoność czasowa wynosi $\boldsymbol{O}(|\boldsymbol{V}| + |\boldsymbol{E}|)$, gdzie $|\boldsymbol{V}|$to liczba wierzchołków, a $|\boldsymbol{E}|$ to liczba krawędzi.[3]

Podobnie jak algorytm BFS, algorytm DFS i Best-First Search posiadają pesymistyczną złożoność czasowa wynoszącą $\boldsymbol{O}(|\boldsymbol{V}| + |\boldsymbol{E}|)$.[4]

---

[1] https://pl.wikipedia.org/wiki/Problem_komiwoja%C5%BCera
[2] chrome-extension://efaidnbmnnnibpcajpcglclefindmkaj/https://ii.uni.wroc.pl/~prz/2011lato/ah/opracowania/met_podz_ogr.opr.pdf
[3] https://pl.wikipedia.org/wiki/Przeszukiwanie_wszerz
[4] https://pl.wikipedia.org/wiki/Przeszukiwanie_w_g%C5%82%C4%85b

Zakładając, że n to liczba wierzchołków, wychodzi nam, że algorytmy mają następującą pesymistyczną złożoność czasową:

$$\boldsymbol{O}\big(\boldsymbol{n}+\boldsymbol{n}(\boldsymbol{n}-\mathbf{1})\big)=\boldsymbol{O}(\boldsymbol{n}+\boldsymbol{n}^{\mathbf{2}})=\boldsymbol{O}(\boldsymbol{n}^{\mathbf{2}})$$

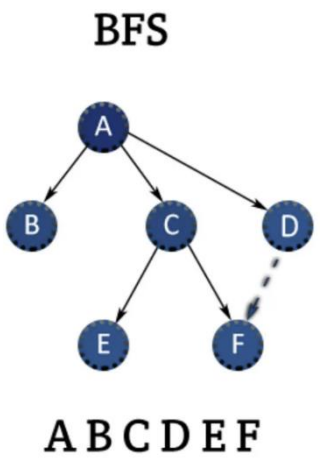
2. Przykład praktyczny

2.1 Algorytm Branch and Bound (przy przeszukiwaniu wszerz BFS)[5]

1. Zainicjalizuj kolejkę.
2. Umieść wierzchołek startowy w kolejce.
3. Dopóki kolejka nie jest pusta:
    a. Dodaj wartość wierzchołka z początku kolejki do zmiennej.
    b. Usuń wierzchołek z kolejki.
4. Sprawdź wszystkie nieodwiedzone wierzchołki.
5. Zredukuj macierz oraz wyznacz koszt dla każdego nieodwiedzonego wierzchołka. Dodaj odpowiedni wierzchołek do ścieżki.
6. Po zakończeniu konstrukcji całej ścieżki, zweryfikuj jej optymalność oraz dodaj wierzchołek początkowy.

Dla algorytm Branch and Bound przy przeszukiwaniu w głąb oraz Best-First Search zmieniono struktury przechowywania na odpowiednio stos oraz kolejkę priorytetową. Dodano również sprawdzenie przed dodaniem wierzchołka czy koszt ścieżki jest większy niż minimalny aktualny koszt sprawdzonych ścieżek.

2.2 Algorytm BFS

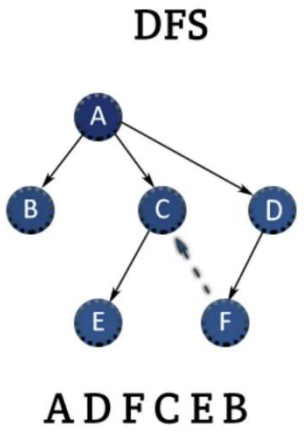


1. Utwórz kolejkę (FIFO) i dodaj do niego węzeł początkowy (A)
2. Oznacz węzeł A jako odwiedzony
3. Dopóki kolejka nie jest pusta:
    a. Usuń węzeł z przodu kolejki (aktualny węzeł), i ustaw go jako węzeł aktualny.
    b. Każdego nieodwiedzonego sąsiada dodaj do kolejki i oznacz go jako nieodwiedzony
4. Po odwiedzeniu wszystkich węzłów zakończ algorytm.

Rysunek 1 Przykład algorytmu BFS
Źródło: https://medium.com/swlh/bfs-vs-dfs-9b644c6c713c

[5] Opracowane na podstawie https://www.youtube.com/watch?v=1FEP_sNb62k oraz algorytmu BFS

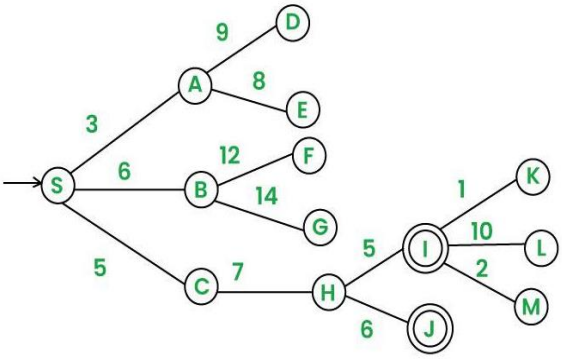
2.3 Algorytm DFS



1. Utwórz stos (LIFO) i dodaj do niego węzeł początkowy (A)
2. Oznacz węzeł A jako odwiedzony
3. Dopóki kolejka nie jest pusta:
   a. Usuń węzeł z przodu kolejki (aktualny węzeł), i ustaw go jako węzeł aktualny.
   b. Dla każdego nieodwiedzonego sąsiada dodaj do kolejki i oznacz go jako nieodwiedzony
4. Po odwiedzeniu wszystkich węzłów zakończ algorytm.

Rysunek 2 Przykład algorytmu DFS
Źródło: https://medium.com/swlh/bfs-vs-dfs-9b644c6c713c

2.4 Algorytm Branch and Bound - Best-First Search



1. Utwórz kolejkę priorytetową, gdzie priorytet to najmniejszy koszt ścieżki
2. Dodaj do kolejki węzeł początkowy
3. Dopóki kolejka nie jest pusta
   a. Usuń węzeł z najwyższym priorytetem, czyli najniższym kosztem i ustaw go jako węzeł aktualny
   b. Jeśli węzeł to cel, zakończ algorytm
4. Dla każdego nieodwiedzonego sąsiada aktualnego węzła dodaj do kolejki priorytetowej i oblicz koszt ścieżki
5. Po odwiedzeniu wszystkich zakończ algorytm

Rysunek 3 Przykład algorytmu Best-First Search
Źródło: https://www.geeksforgeeks.org/best-first-search-informed-search/

6. Plan eksperymentu

a. Rozmiary używanych struktur danych

Będą generowane wyniki dla 7 reprezentatywnych rozmiarów macierzy: 3,4,5,6,7,8,9. Macierze będą symetryczne i asymetryczne.

Macierz jest przechowywana w obiekcie klasy Matrix. Obiekt posiada atrybuty rozmiar, wskaźnik do tablicy dwuwymiarowej z wartościami z macierzy oraz informacje czy macierz jest symetryczna, czy asymetryczna.

Oprócz macierzy wykorzystujemy również strukturę myNode, która zawiera pola numer miasta, koszt ścieżki, wektor ze ścieżką oraz zredukowaną macierz.

Struktury takie jak: kolejka, kolejka priorytetowa, stos były wykorzystane z gotowych bibliotek ze względu na słabsze wyniki działania algorytmu dla napisanych struktur.

b. Sposób generowania danych

Macierz możemy wygenerować na dwa sposoby: odczytanie z pliku oraz wygenerowanie macierzy losowej.

- *Odczytanie z pliku:* W tym przypadku wystarczy podać nazwę pliku znajdującego się w katalogu „files". Macierz zostanie odczytana z tego pliku.
- *Generowanie macierzy losowej:* Wymaga podania rozmiaru macierzy. Powstała macierz będzie dynamiczną, dwuwymiarową tablicą wypełnioną wartością „-1". Następnie musimy wypełnić macierz losowymi wartościami. Jeśli chcemy, aby macierz była asymetryczna, używamy funkcji „fillRandom()". Dla macierzy symetrycznej korzystamy z funkcji „fillRandomSymmetric()".

c. Metoda pomiaru czasu

W celu pomiaru czasu wykonania algorytmu użyto funkcji QueryPerformanceCounter() oraz QueryPerformanceFrequency(), które umożliwiają uzyskanie dokładnego wyniku dzięki wysokiej precyzji dostępnej w systemach Windows. Pomiary zostały wykonane w mikrosekundach.

Działanie funkcji jest następujące:

1. Na początku uruchamiany jest stoper (startTimer()) za pomocą funkcji QueryPerformanceCounter(), która zapisuje aktualną wartość licznika wydajności do zmiennej „start".
2. Po zakończeniu operacji, stoper jest zatrzymywany (stopTimer()) i zapisywana jest wartość „end".
3. Funkcja diffTime() oblicza różnicę pomiędzy wartościami „start" i „end", aby uzyskać czas wykonania operacji. W tym celu wykorzystywana jest także funkcja QueryPerformanceFrequency(), która podaje częstotliwość licznika.
4. Różnica difference.QuadPart jest przeliczana na mikrosekundy przez pomnożenie przez 1 000 000 i podzielenie przez częstotliwość.

Podczas testowania zapisuje się czas rozpoczęcia działania, a po zakończeniu wykonania obliczana jest różnica czasu. Czas ten jest sumowany w zmiennej „sred" w celu obliczenia średniego czasu działania dla kilku iteracji.

d. Warunki badawcze:

a. specyfikacja sprzętu:
   i. procesor: AMD Ryzen 7 7840HS Radeon 780M Graphics
   ii. zegar: 3801 Mhz
   iii. wielkość pamięci RAM: 32 GB

b. Badania były przeprowadzone na prezentowanym sprzęcie. Komputer pracował na zasilaniu sieciowym. Proces nie był przypisany do żadnego z rdzeni procesora, ani nie miał ustalonej częstotliwości taktowania.

7. Wyniki eksperymentu

a. Macierze symetryczne

Tabela 1 Tabela pokazuje czasy wykonania trzech różnych algorytmów rozwiązujących problem komiwojażera (TSP) dla różnych wielkości instancji n miast dla macierzy symetrycznych.

Symetryczne			
	BFS	DFS	bestFS
n	**Czas [ms]**		
3	9	9	12
4	37	26	50
5	191	76	163
6	1076	215	405
7	7123	524	1111
8	55208	1352	3147
9	517990	3137	7578

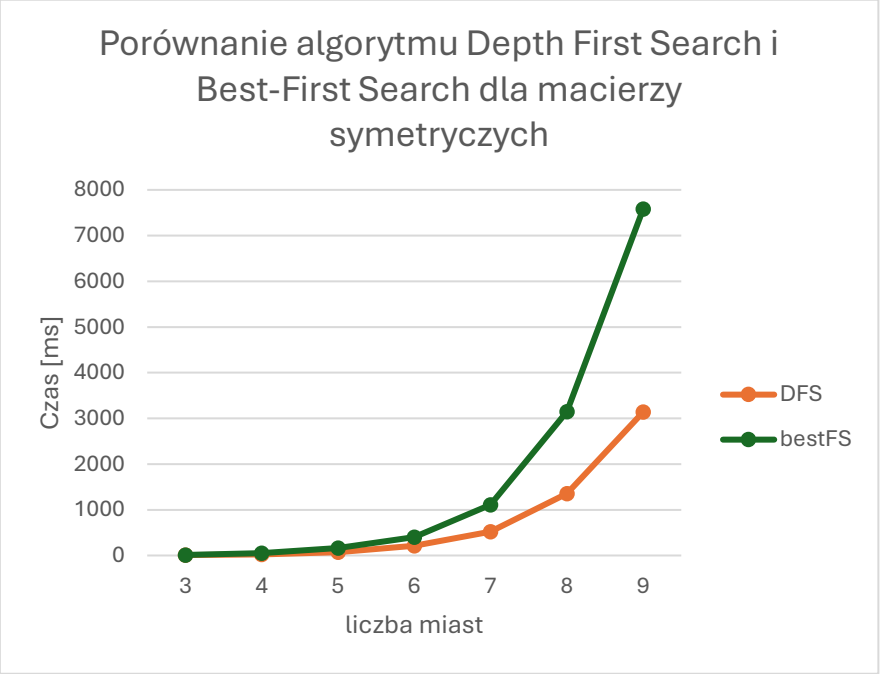
Źródło: Opracowanie własne

Wyniki pokazują, że czas działania wszystkich trzech algorytmów (BFS, DFS, BestFS) wzrasta wraz z rozmiarem instancji, jednak charakter tego wzrostu różni się między algorytmami. Dla małych wartości (np. pierwsze trzy wiersze), różnice w czasach działania są niewielkie — BFS i DFS mają porównywalne wyniki, a BestFS działa nieco wolniej.

Przy większych rozmiarach problemu różnice stają się wyraźniejsze. Przykładowo, dla piątego wiersza (instancja 7123 ms dla BFS) czas działania DFS wynosi 524 ms, a BestFS 1111 ms, co wskazuje na znacznie lepszą skalowalność DFS.

Dla największych instancji różnice czasowe są jeszcze bardziej znaczące. BFS wymaga aż 63,7 miliona milisekund, podczas gdy DFS i BestFS odpowiednio 18,3 miliona oraz 58,2 miliona. DFS zachowuje przewagę wydajności nawet przy dużej liczbie operacji, co czyni go najbardziej efektywnym w tej grupie.

Rysunek 4 Porównanie algorytmów dla macierzy symetrycznych
Źródło: Opracowanie własne



Rysunek 5 Porównywanie algorytmu DFS i Best-First Search dla macierzy symetrycznych
Źródło: Opracowanie własne

b. Macierze asymetryczne

Tabela 2 Tabela pokazuje czasy wykonania trzech różnych algorytmów rozwiązujących problem komiwojażera (TSP) dla różnych wielkości instancji n miast dla macierzy symetrycznych.

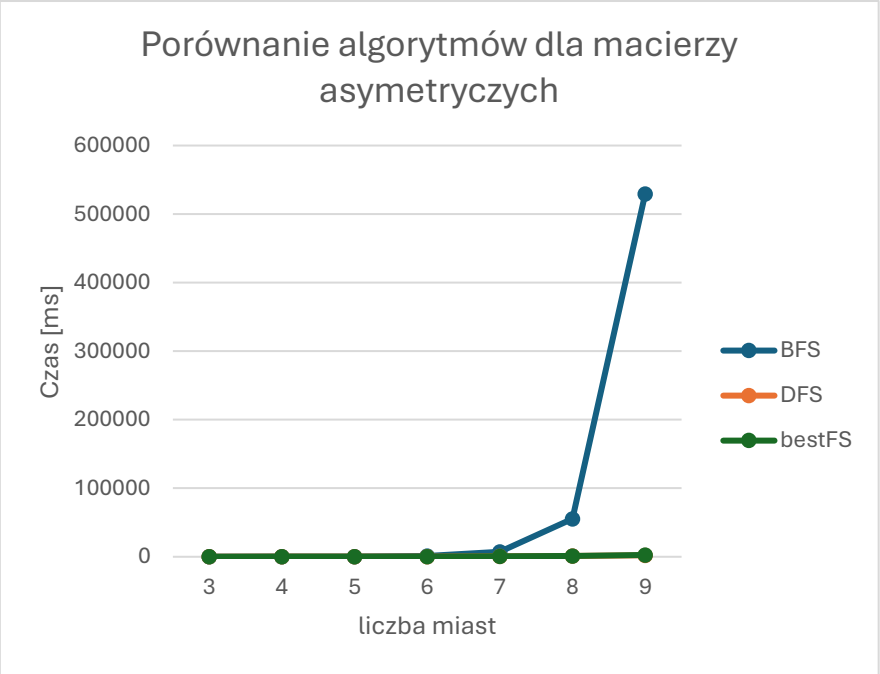
Asymetryczne			
	BFS	DFS	bestFS
n	**Czas [ms]**		
3	15	14	20
4	38	28	38
5	197	82	106
6	1076	196	254
7	7023	479	522
8	55335	1019	1077
9	529627	2220	2580

Źródło: Opracowanie własne

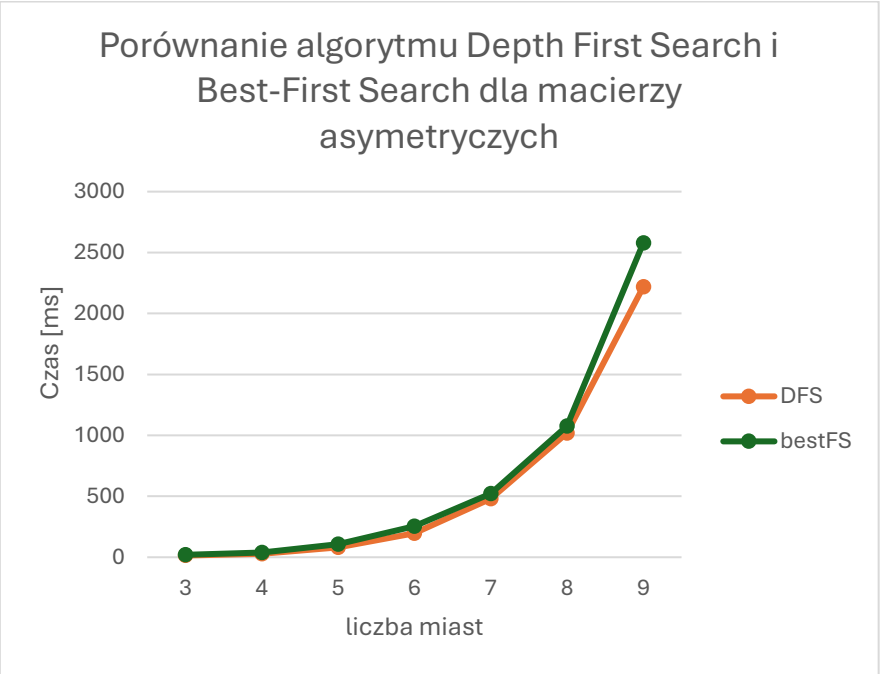
Wyniki pokazują, że czas działania wszystkich trzech algorytmów wzrasta wraz z rozmiarem instancji, jednak BFS wymaga więcej czasu w porównaniu do DFS i BestFS, co wynika z większej liczby operacji przeszukiwania. Dla małych rozmiarów (np. pierwsze dwa wiersze), różnice między algorytmami są minimalne — BFS, DFS i BestFS działają zbliżonym tempem, z czasami w zakresie 15–20 ms.

Przy większych instancjach różnice stają się bardziej zauważalne. Przykładowo, dla średniego rozmiaru instancji (7023 ms dla BFS) algorytm DFS wymaga jedynie 479 ms, a BestFS 522 ms, co oznacza, że BFS jest ponad 14 razy wolniejszy od DFS i około 13,5 razy wolniejszy od BestFS.

Dla największych instancji różnice czasowe gwałtownie rosną. Algorytm BFS wymaga aż 529627 ms, podczas gdy DFS działa w czasie 2220 ms, a BestFS w 2580 ms. Oznacza to, że BFS jest ponad 230 razy wolniejszy od DFS i około 205 razy wolniejszy od BestFS. Różnica między DFS i BestFS dla dużych rozmiarów wynosi jedynie 360 ms, co wskazuje na ich porównywalną wydajność w tych warunkach.



Rysunek 6 Porównanie algorytmów dla macierzy symetrycznych
Źródło: Opracowanie własne



Rysunek 7 Porównywanie algorytmu DFS i Best-First Search dla macierzy asymetrycznych
Źródło: Opracowanie własne

8. Wnioski

a. Tabela symetryczna

Na podstawie tabeli nr. 1 możemy wywnioskować, że algorytm Depth First Search okazał się najbardziej efektywny, szczególnie w dużych instancjach, gdzie różnice w czasach działania między algorytmami były najbardziej zauważalne. Algorytm Breadth First Search działał dobrze dla małych problemów, jednak jego czas obliczeń wzrastał wykładniczo wraz z rozmiarem danych, co czyniło go najmniej praktycznym dla większych instancji. Best-First Search, choć szybszy niż Breadth First Search, pozostawał znacznie wolniejszy od Depth First Search w każdym z analizowanych przypadków. Depth First Search wykazał najlepszą skalowalność i był najbardziej efektywnym wyborem dla macierzy symetrycznych.

b. Tabela asymetryczna

Analizując wyniki w tabeli nr. 2 W macierzach asymetrycznych algorytmy Depth First Search i Best-First Search osiągały bardzo podobne wyniki, szczególnie dla dużych instancji, gdzie różnice w czasach działania były marginalne. Breadth First Search, podobnie jak w przypadku macierzy symetrycznych, okazał się najmniej efektywny, choć różnice między nim a Depth First Search oraz Best-First Search były mniejsze niż w macierzach symetrycznych. Depth First Search i Best-First Search, dzięki swojej porównywalnej wydajności, można uznać za najbardziej efektywne w przetwarzaniu macierzy asymetrycznych.

c. Porównanie Depth First Search i Best-First Search dla obu typów macierzy

Jeśli porównamy wyniki z rysunku nr. 5 i 7, możemy zauważyć, że w macierzach symetrycznych Depth First Search wyraźnie przewyższa Best-First Search, szczególnie w przypadku dużych instancji, gdzie różnice w czasach działania są znaczące. Natomiast w macierzach asymetrycznych Depth First Search i Best-First Search działają zbliżenie efektywnie, z minimalnymi różnicami czasowymi, co sprawia, że oba algorytmy można uznać za równoważne w tych scenariuszach.

d. Efektywność poszczególnych algorytmów

Breadth First Search może być uznany za efektywny wyłącznie w przypadku bardzo małych instancji, gdzie różnice w czasach działania między algorytmami są niewielkie. Jednak wraz ze wzrostem rozmiaru problemu jego czas działania rośnie najszybciej, co sprawia, że staje się niepraktyczny. Można to zauważyć na obu rysunkach nr. 4 i nr. 6. Jeśli spojrzymy zaś na rysunek nr. 5 i nr. 7, Depth First Search jest najbardziej efektywny w macierzach symetrycznych, niezależnie od rozmiaru danych, oraz w asymetrycznych macierzach, gdzie osiąga wyniki porównywalne z Best-First Search. Algorytm Best-First Search najlepiej sprawdza się w asymetrycznych problemach, gdzie jego wydajność jest porównywalna z Depth First Search, choć w macierzach symetrycznych pozostaje mniej efektywny.

e. Możliwe poprawki

Ze względu na zastosowane struktury, implementacja algorytmu Best-First Search mogłaby zostać zrealizowana w sposób bardziej optymalny i efektywny. Niestety, prace nad tym algorytmem zostały zrealizowane w końcowej fazie projektu, co sprawiło, że zauważone możliwości usprawnień pojawiły się zbyt późno, by wprowadzić odpowiednie poprawki.

Podsumowując:

Algorytm Depth First Search jest uniwersalnym wyborem, szczególnie w sytuacjach wymagających przetwarzania dużych instancji, gdzie jego skalowalność i czas działania wyraźnie przewyższają inne metody.

Algorytm Best-First Search stanowi dobrą alternatywę w przypadku asymetrycznych macierzy, a **Breadth First Search** jest odpowiedni jedynie dla małych problemów, gdzie jego ograniczenia czasowe nie odgrywają kluczowej roli. Wybór algorytmu powinien uwzględniać zarówno charakterystykę problemu (symetria/asymetria), jak i jego rozmiar.

9. Kod źródłowy

Zostanie załączony wraz ze sprawozdaniem.

10. Bibliografia:

1. https://pl.wikipedia.org/wiki/Problem_komiwoja%C5%BCera
2. chrome-extension://efaidnbmnnnibpcajpcglclefindmkaj/https://ii.uni.wroc.pl/~prz/2011lato/ah/opracowania/met_podz_ogr.opr.pdf
3. https://pl.wikipedia.org/wiki/Przeszukiwanie_wszerz
4. https://pl.wikipedia.org/wiki/Przeszukiwanie_w_g%C5%82%C4%85b
5. https://en.wikipedia.org/wiki/Best-first_search
6. https://www.youtube.com/watch?v=1FEP_sNb62k